

C-C++\Projects\cont.cpp

```

1  #include<iostream>
2  #include<sstream>
3  #include<fstream>
4  #include<vector>
5  #include<algorithm>
6  #include<iomanip>
7
8  using namespace std;
9
10 // FILE NAME THAT WILL BE USED IN THE PROGRAM. CAN BE CHANGED ANYTIME BY MODIFYING THE
    NAME BETWEEN BRACKETS.
11 #define DATA_FILE "virtual.csv"
12
13 // A STRUCT TO HANDLE THE NAME AND THE NUMBER. CAN BE UPGRADED EASILY TO CONTAIN MORE
    FIELDS.
14 typedef struct
15 {
16     string name;    // CONTACT NAME.
17     string number;  // CONTACT NUMBER.
18 }cntct;
19
20 // ARRAY CONTAINING THE SERVICES NAMES THAT THE PROGRAM PROVIDES.
21 const string MENU_ITEMS[] =
22 {
23     "WELCOME TO YOUR PHONE BOOK :\n",    // WELCOME MESSAGE.
24     "(1). VIEW ALL CONTACTS.",           // OPTION TO VIEW ALL CONTACTS.
25     "(2). ADD NEW CONTACT.",             // OPTION TO ADD A NEW CONTACT.
26     "(3). SEARCH YOUR PHONE BOOK.",      // OPTION TO SEARCH CONTACTS.
27     "(4). EXIT PROGRAM."                 // OPTION TO EXIT THE PROGRAM.
28 };
29
30 // SEPARATOR LINE USED FOR FORMATTING OUTPUT.
31 const string line = "-----\n";
32
33 // FILE STREAM FOR THE USED .CSV FILE.
34 fstream FS;
35
36 // VECTOR THAT WILL CONTAIN CONTACTS SAVED IN THE .CSV FILE.
37 vector <cntct> CONATCTS;
38
39 // FUNCTION DECLARATIONS.
40 void MAIN_MENU(void);           // FUNCTION TO PRINT THE MAIN MENU ITEMS.
41 void VIEW_ALL_CONTACTS(void);   // FUNCTION TO VIEW ALL SAVED CONTACTS.
42 void ADD_NEW_CONTACT(void);     // FUNCTION TO ADD A NEW CONTACT.
43 void SEARCH_CONTACTS(void);     // FUNCTION TO SEARCH THE PHONEBOOK.
44 string GET_STRING (size_t max_length); // FUNCTION TO GET A STRING FROM THE USER.
45 void CLEAR_SCREEN(void);        // FUNCTION TO CLEAR THE SCREEN.
46 void ADD_CONTACTS_TO_VECTOR (void); // FUNCTION TO ADD .CSV DATA TO THE VECTOR.
47 void SORT_VECTOR_AND_FILE (void); // FUNCTION TO SORT VECTOR ELEMENTS AND REWRITE THEM
    TO THE FILE SORTED.
48 void FILE_CONTROL_OPEN_AND_CLOSE(char Control); // FUNCTION TO OPEN OR CLOSE THE FILE.
49 void PRINT_OUT_INPUT(cntct C);  // FUNCTION TO PRINT CONTACT DETAILS BEFORE SAVING.

```

```

50
51 int main()
52 {
53     // OPENING FILE FOR READING AND WRITING.
54     FILE_CONTROL_OPEN_AND_CLOSE('o');
55     // ADDING CONTACTS TO VECTOR FOR USE IN THE PROGRAM FLOW.
56     ADD_CONTACTS_TO_VECTOR();
57     // SORTING THE DATA FROM THE FILE IF NOT ALREADY SORTED.
58     SORT_VECTOR_AND_FILE();
59     while(true)
60     {
61         // PRINTING THE MAIN MENU OPTIONS.
62         MAIN_MENU();
63         int choice;
64         cin>>choice;
65
66         // INPUT ERROR HANDLING.
67         if(cin.fail())
68         {
69             cin.clear();
70             cin.ignore();
71             cout<<"RE-CHOOSE THE PREFERRED SERVICE NUMBER";
72         }
73
74         switch(choice)
75         {
76             case 1:// (1.) VIEW ALL CONTACTS.
77             {
78                 CLEAR_SCREEN(); // CLEAR TERMINAL.
79                 cout<<MENU_ITEMS[1]<<endl; // PRINTING THE CHOSEN OPTION HEADLINE.
80                 cout<<line<<endl;
81                 VIEW_ALL_CONTACTS();
82                 cout << "\nPRESS ENTER TO RETURN TO THE MAIN MENU...";
83                 cin.ignore(); // REMOVE ANY REMAINING DATA IN THE BUFFER (IMPORTANT
BEFORE cin.get()).
84                 cin.get();    // WAIT FOR THE USER TO PRESS ENTER.
85                 CLEAR_SCREEN();
86             }
87             break;
88
89             case 2:// (2.) ADD NEW CONTACT.
90             {
91                 CLEAR_SCREEN(); // CLEAR TERMINAL.
92                 cout<<MENU_ITEMS[2]<<endl<<line; // PRINTING THE CHOSEN OPTION HEADLINE.
93                 ADD_NEW_CONTACT(); // ADDING A NEW CONTACT USING THE FUNCTION DEFINED
BELOW.
94             }
95             break;
96
97             case 3:// (3.) SEARCH CONTACTS.
98             {
99                 CLEAR_SCREEN(); // CLEAR TERMINAL.
100                 cout<<MENU_ITEMS[3]<<endl<<line; // PRINTING THE CHOSEN OPTION HEADLINE.
101                 SEARCH_CONTACTS(); // SEARCHING USING THE FUNCTION DEFINED BELOW.

```

```

102     }
103     break;
104
105     case 4:// (4.) EXIT PROGRAM.
106     {
107         CLEAR_SCREEN(); // CLEAR TERMINAL.
108         cout<<"SEE YOU SOON"<<endl<<line; // PRINTING THE CHOSEN OPTION HEADLINE.
109         FILE_CONTROL_OPEN_AND_CLOSE('c'); // CLOSING THE FILE SAFELY.
110         return 0;
111     }
112     break;
113
114     default:
115     {
116         CLEAR_SCREEN(); // CLEAR TERMINAL.
117         cout<<"RE-CHOOSE THE PREFERRED SERVICE NUMBER"; // INPUT ERROR HANDLING.
118         break;
119     }
120 }
121 }
122 return 0;
123 }
124
125 void MAIN_MENU (void)
126 {
127     CLEAR_SCREEN(); // CLEAR TERMINAL.
128
129     // SIZE OF THE GLOBAL ARRAY ABOVE^^^.
130     unsigned int MENU_ITEMS_NUMBER = sizeof(MENU_ITEMS)/sizeof(MENU_ITEMS[0]);
131     // A LOOP TO PRINT ALL THE MENU ITEMS VERTICALLY.
132     for(unsigned int i = 0; i < MENU_ITEMS_NUMBER; i++)
133     {
134         cout<<MENU_ITEMS[i]<<endl;
135     }
136     // INSTRUCTION MESSAGE.
137     cout<< "\n"<<"FORM 1"<< " TO "<< MENU_ITEMS_NUMBER -1 <<endl;
138     cout<<"ENTER PREFERRED CHOICE NUMBER THEN PRESS ""ENTER"" >> ";
139 }
140
141 void CLEAR_SCREEN(void)
142 {
143     #ifdef _WIN32
144         system("cls"); // COMPATIBLE WITH WINDOWS.
145     #else
146         system("clear"); // COMPATIBLE WITH LINUX & macOS.
147     #endif
148 }
149
150 void VIEW_ALL_CONTACTS(void)
151 {
152     // SIMPLY PRINTING CONTACTS SAVED IN THE VECTOR.
153     for (const auto& c : CONATCTS)
154     {
155         cout << left << setw(25) << c.name << setw(15) << c.number << endl;

```

```

156     }
157 }
158
159 void ADD_CONTACTS_TO_VECTOR (void)
160 {
161     // READING LINE BY LINE VERTICALLY.
162     string READING_LINE;
163     while(getline(FS,READING_LINE))
164     {
165         // AVOIDING THE HEADER LINE IN THE .CSV FILE.
166         if(READING_LINE.empty() || READING_LINE.rfind("name",0) == 0)
167         {
168             continue;
169         }
170         // INITIALIZE NAME AND NUMBER FOR TWO STRINGS TO STORE THEM SEPARATELY IN THE
171         VECTOR OF CNTCT.
172         string NAME,NUMBER;
173         // STRING STREAM TO SEPARATE THE TEXT BEFORE AND AFTER THE ','.
174         stringstream STRNG_LN (READING_LINE);
175         // STORING DATA BEFORE ',' IN THE NAME FIELD.
176         getline(STRNG_LN , NAME , ',');
177         // STORING DATA AFTER ',' IN THE NUMBER FIELD.
178         getline(STRNG_LN , NUMBER);
179         // ADD EACH CONTACT NAME AND NUMBER TO THE VECTOR.
180         CONATCTS.push_back({NAME,NUMBER});
181     }
182 }
183
184 void SORT_VECTOR_AND_FILE (void)
185 {
186     // CLOSE THE FILE FIRST BEFORE OPENING IT WITH `TRUNC` MODE.
187     if (FS.is_open())
188     {
189         FS.close();
190     }
191     // RE-OPENING THE FILE TO REWRITE SORTED DATA.
192     FS.open(DATA_FILE, ios::out | ios::trunc);
193     // FILE OPENING ERROR HANDLING.
194     if (!FS)
195     {
196         cout << "ERROR OPENING FILE IN SORT_VECTOR_AND_FILE\n";
197         return;
198     }
199     // SORTING CONTACTS.
200     sort(CONATCTS.begin(),CONATCTS.end(),[](const cntct& a,cntct& b)
201     {
202         return a.name < b.name;
203     });
204     // REWRITE THE DATA AFTER SORTING IN THE FILE.
205     for ( auto c : CONATCTS)
206     {
207         FS<<c.name<<','<<c.number<<endl;
208     }
209 }

```

```

209
210 void FILE_CONTROL_OPEN_AND_CLOSE(char Control)
211 {
212     // CHECKING FUNCTION ARGUMENTS.
213     if(Control == 'o' || Control == 'O')
214     {
215         FS.open(DATA_FILE, std::ios::in | std::ios::out | std::ios::app);
216         if (!FS)
217         {
218             cout<<"ERROR OPENING FILE IN FILE_CONTROL_OPEN_AND_CLOSE FUNCTION";
219             return;
220         }
221     }
222     // CHECKING FUNCTION ARGUMENTS.
223     else if(Control == 'c' || Control == 'C')
224     {
225         {
226             if (FS.is_open()) // CHECKING IF FILE IS STILL OPEN.
227             {
228                 FS.close();
229             }
230             else
231             {
232                 cout<<"FILE IS NOT OPENED";
233             }
234         }
235     }
236 }
237 else
238 {
239     // ERROR MESSAGE IF SOMETHING IS WRONG.
240     cout<<"ERROR USING ""FILE_CONTROL_OPEN_AND_CLOSE"" FUNCTION ";
241 }
242 }
243
244 void ADD_NEW_CONTACT (void)
245 {
246     // DECLARING A CONTAINER TO CONTAIN THE NAME AND NUMBER.
247     cntct NEW;
248
249     // PROMPT USER FOR NAME INPUT.
250     cout<<"ENTER CONTACT NAME : ";
251     NEW.name = GET_STRING(20); // GETTING INPUT USING THE GET_STRING FUNCTION.
252
253     // PROMPT USER FOR NUMBER INPUT.
254     cout<<"ENTER CONTACT NUMBER : ";
255     NEW.number = GET_STRING(12); // GETTING INPUT USING THE GET_STRING FUNCTION.
256
257     CLEAR_SCREEN(); // CLEAR TERMINAL.
258     // PRINTING OUT THE INPUT TO LET THE USER CHECK IT BEFORE SAVING.
259     PRINT_OUT_INPUT(NEW);
260     cout<<line<<endl;
261
262     // PRINTING OUT OPTIONS FOR THE INPUT.

```

```

263     string msg ="1 - CONFIRM SAVING\n2 - RE-ENTER DETAILS\n3 - EXIT TO MAIN MENU WITHOUT
SAVING >>> ";
264     cout<<msg;
265
266     int Confirm;
267     do
268     {
269         cin >> Confirm;
270         // INPUT ERROR HANDLING.
271         if(cin.fail())
272         {
273             // CLEAR THE ERROR FLAG.
274             cin.clear();
275             // IGNORE INVALID INPUT UNTIL THE END OF THE LINE.
276             cin.ignore(10000, '\n');
277             // PRINT ERROR MESSAGE.
278             CLEAR_SCREEN();
279             cout << "INVALID CHOICE TRY AGAIN" << endl;
280             // PRINT OUT THE INPUT AGAIN.
281             PRINT_OUT_INPUT(NEW);
282             cout<<line;
283             cout << msg << endl;
284         }
285         else
286         {
287             // CHOOSING THE PREFERRED CHOICE.
288             switch(Confirm)
289             {
290                 case 1:// CONFIRM SAVING.
291                 {
292                     // ADD THE NEW CONTACT TO THE VECTOR.
293                     CONATCTS.push_back({NEW.name,NEW.number});
294                     // REWRITE THE VECTOR TO THE FILE AFTER SAVING AND SORTING.
295                     SORT_VECTOR_AND_FILE();
296                     CLEAR_SCREEN(); // CLEAR TERMINAL.
297                     cout<<"CONTACT SAVED SUCCESSFULLY PRESS ENTER TO CONTINUE...";
298                     // REMOVE ANY REMAINING DATA IN THE BUFFER (IMPORTANT BEFORE
cin.get()).
299                     cin.ignore();
300                     // WAIT FOR THE USER TO PRESS ENTER.
301                     cin.get();
302                     CLEAR_SCREEN(); // CLEAR TERMINAL.
303                     return;
304                 }
305                 break;
306
307                 case 2:// RE-ENTER DETAILS.
308                 {
309                     // PRINT OUT THE HEADLINE.
310                     cout<<MENU_ITEMS[2];
311                     CLEAR_SCREEN(); // CLEAR TERMINAL.
312                     ADD_NEW_CONTACT(); // RECURSION OF THE FUNCTION UNTIL SAVING OR
REFUSING.
313                     }break;

```

```

314
315     case 3:// BACK TO MAIN MENU.
316     {
317         CLEAR_SCREEN();
318         return;
319     }
320     break;
321
322     default:
323     {
324         // INPUT ERROR HANDLING.
325         CLEAR_SCREEN();
326         cout<<"INVALID CHOICE TRY AGAIN"<<endl;
327         PRINT_OUT_INPUT(NEW);
328         cout<<line;
329         cout<<msg;
330     }
331 }
332 }
333 }while(Confirm !=3);
334 }
335
336 string GET_STRING (size_t max_length)
337 {
338     string String;
339
340     // IF THE NEXT CHARACTER IS A NEWLINE, THEN IGNORE IT.
341     if(cin.peek() == '\n')
342     {
343         cin.ignore(numeric_limits<streamsize>::max(), '\n');
344     }
345     // PROMPT USER FOR INPUT.
346     getline(cin,String);
347     // TRUNCATING THE STRING TO GET THE PREFERRED LENGTH.
348     if((String).length() > max_length && (String).length() != 0)
349     {
350         String = (String).substr(0,max_length);
351     }
352     // CONVERT ALL CHARACTERS TO LOWERCASE.
353     transform(String.begin() , String.end() , String.begin() , ::tolower);
354     // IF THE STRING IS NOT EMPTY AND THE FIRST CHARACTER IS ALPHABETIC, CAPITALIZE IT.
355     if (!String.empty() && isalpha(String[0]))
356     {
357         String[0] = toupper(String[0]);
358     }
359
360     // FOR EACH CHARACTER STARTING FROM THE SECOND,
361     // IF THE PREVIOUS CHARACTER IS A SPACE AND THE CURRENT CHARACTER IS ALPHABETIC,
362     CAPITALIZE IT.
363     for (size_t i = 1; i < String.size(); i++)
364     {
365         if (String[i - 1] == ' ' && isalpha(String[i]))
366         {
367             String[i] = toupper(String[i]);

```

```

367         }
368     }
369     return String;
370 }
371
372 void SEARCH_CONTACTS(void)
373 {
374     // PROMPT USER TO CHOOSE SEARCH CRITERIA (BY NAME OR NUMBER).
375     int searchChoice;
376     do
377     {
378         CLEAR_SCREEN();
379         cout << "SEARCH BY:\n"<<line<<"1. NAME\n2. NUMBER\n\n";
380         cout << "ENTER YOUR CHOICE (1 OR 2): ";
381         cin >> searchChoice;
382
383         // HANDLE INVALID INPUT.
384         if(cin.fail())
385         {
386             cin.clear();
387             cin.ignore(10000,'\n');
388             cout << "INVALID INPUT!\n";
389         }
390     }while(searchChoice !=1 && searchChoice != 2);
391
392     // PROMPT USER FOR SEARCH TERM.
393     string searchTerm;
394     cout << "ENTER SEARCH TERM: ";
395     cin.ignore();
396     getline(cin, searchTerm);
397
398     // CONVERT SEARCH TERM TO LOWERCASE FOR CASE-SENSITIVE SEARCH.
399     transform(searchTerm.begin(), searchTerm.end(), searchTerm.begin(), ::tolower);
400
401     // SEARCH FOR MATCHING CONTACTS.
402     bool found = false;
403     CLEAR_SCREEN();
404     cout << "\nSEARCH RESULTS FOR :"<<searchTerm<<endl;
405     cout << line << endl;
406
407     for(const auto& contact : CONATCTS)
408     {
409         string compareName = contact.name;
410         transform(compareName.begin(), compareName.end(), compareName.begin(), ::tolower);
411
412         // CHECK IF SEARCH TERM MATCHES NAME OR NUMBER BASED ON USER CHOICE.
413         if(searchChoice == 1 && compareName.find(searchTerm) != string::npos)
414         {
415             cout << left << setw(25) << contact.name << setw(15) << contact.number <<
416             endl;
417             found = true;
418         }
419         else if(searchChoice == 2 && contact.number.find(searchTerm) != string::npos)
420         {

```

```
420         cout << left << setw(25) << contact.name << setw(15) << contact.number <<
endl;
421         found = true;
422     }
423 }
424
425 // PRINT MESSAGE IF NO MATCHES FOUND.
426 if(!found)
427 {
428     cout << "NO CONTACTS FOUND MATCHING YOUR SEARCH.\n";
429 }
430
431 cout << line << endl;
432 cout << "\nPRESS ENTER TO RETURN TO THE MAIN MENU...";
433 cin.get();
434 CLEAR_SCREEN();
435 }
436
437 void PRINT_OUT_INPUT(cntct C)
438 {
439     // PRINT THE CONTACT DETAILS BEFORE SAVING.
440     cout<<"YOU ARE ABOUT TO SAVE TO YOUR CONTACTS :\n"<<line;
441     cout<<"NAME : "<<C.name<<endl;
442     cout<<"NUMBER : "<<C.number<<endl;
443 }
```